

SISTEMA DE MONITORAMENTO DE HORTA INTELIGENTE

Documentação de Arquitetura de Banco de Dados

Stack tecnológica:

ESP32 · DHT22 · OpenCV · Python · MQTT · Spring Boot · PostgreSQL · Angular

Versão 1.0 · 2025

1. Introdução

Este documento descreve a arquitetura do banco de dados do Sistema de Monitoramento de Horta Inteligente, desenvolvido como projeto integrador. O sistema combina hardware embarcado, visão computacional, mensageria IoT e uma aplicação web de tempo real para monitorar e controlar automaticamente o cultivo de plantas.

O banco de dados PostgreSQL é o repositório central de todas as informações coletadas pelos dispositivos ESP32, analisadas pelo módulo OpenCV/Python e exibidas pelo frontend Angular. O schema foi projetado para suportar:

- Histórico completo de leituras de sensores (temperatura, umidade, solo)
- Rastreabilidade de eventos de irrigação automática e manual
- Armazenamento e vínculo de imagens com análises de visão computacional
- Sistema de alertas automáticos baseado em parâmetros por espécie de planta
- Log de auditoria de todos os eventos do sistema

2. Visão Geral da Arquitetura

A tabela abaixo apresenta as camadas tecnológicas do sistema e suas responsabilidades dentro da arquitetura geral:

Camada	Tecnologia	Responsabilidade
Hardware / Sensores	ESP32 + DHT22 + Sensor solo	Coleta de dados ambientais e controle da bomba de irrigação
Mensageria	MQTT Broker	Transporte leve de mensagens entre ESP32 e backend (protocolo IoT padrão)
Visão computacional	OpenCV + Python	Análise automática das imagens capturadas pela câmera Canon
Backend / API	Spring Boot (Java)	API REST + persistência + lógica de negócio + publicação MQTT
Banco de dados	PostgreSQL	Armazenamento relacional de todos os dados do sistema
Frontend	Angular	Dashboard web em tempo real com gráficos, alertas e controle remoto

O fluxo principal de dados segue o caminho: Sensor → ESP32 → MQTT Broker → Spring Boot → PostgreSQL → Angular (via WebSocket). O módulo de visão computacional opera de forma assíncrona: a câmera Canon captura imagens, o OpenCV as processa em Python e os resultados são persistidos no banco e exibidos no dashboard.

3. Descrição das Entidades

3.1 Tabela: plant

Catálogo das espécies vegetais monitoradas. Armazena os parâmetros ideais de cada planta, usados como referência para gerar alertas automáticos quando os sensores detectam valores fora da faixa.

Campo	Tipo	Restrição	Null	Descrição
id	UUID	PK	NÃO	Identificador único da planta (gerado automaticamente)
common_name	VARCHAR(100)	—	NÃO	Nome popular da planta (ex: Alface, Manjeriç�o)
scientific_name	VARCHAR(150)	—	SIM	Nome cient�fico (ex: Lactuca sativa)
description	TEXT	—	SIM	Descri��o geral da esp�cie e caracter�sticas de cultivo
optimal_temp_min	FLOAT	—	N�O	Temperatura m�nima ideal em graus Celsius
optimal_temp_max	FLOAT	—	N�O	Temperatura m�xima ideal em graus Celsius
optimal_humidity_min	FLOAT	—	N�O	Umidade relativa do ar m�nima ideal (%)
optimal_humidity_max	FLOAT	—	N�O	Umidade relativa do ar m�xima ideal (%)
optimal_soil_moisture_min	FLOAT	—	N�O	Umidade m�nima do solo ideal (%)
optimal_soil_moisture_max	FLOAT	—	N�O	Umidade m�xima do solo ideal (%)
watering_interval_hours	INTEGER	—	N�O	Intervalo recomendado entre irriga��es (em horas)
created_at	TIMESTAMP	—	N�O	Data e hora de cadastro do registro

3.2 Tabela: device

Representa cada dispositivo ESP32 implantado na horta. O MAC address serve como identificador f sico  nico. O campo last_seen_at   atualizado via heartbeat MQTT e permite detectar dispositivos offline.

Campo	Tipo	Restri��o	Null	Descri��o
id	UUID	PK	N�O	Identificador �nico do dispositivo
plant_id	UUID	FK	N�O	Refer�ncia � planta monitorada por este dispositivo

Campo	Tipo	Restrição	Null	Descrição
name	VARCHAR(80)	—	NÃO	Nome amigável do dispositivo (ex: "ESP32-Horta-01")
mac_address	VARCHAR(17)	—	NÃO	Endereço MAC físico do ESP32 (formato AA:BB:CC:DD:EE:FF)
firmware_version	VARCHAR(20)	—	SIM	Versão do firmware instalado (ex: 1.2.3)
location_description	VARCHAR(200)	—	SIM	Descrição textual da localização física do dispositivo
is_active	BOOLEAN	—	NÃO	Indica se o dispositivo está ativo no sistema
last_seen_at	TIMESTAMP	—	SIM	Último heartbeat recebido via MQTT
created_at	TIMESTAMP	—	NÃO	Data e hora de cadastro do dispositivo

3.3 Tabela: sensor_reading

Armazena cada leitura dos sensores físicos (DHT22 para temperatura/umidade e sensor capacitivo para umidade do solo). O campo status é calculado no momento da inserção comparando os valores lidos com os parâmetros ideais da planta vinculada ao dispositivo.

Campo	Tipo	Restrição	Null	Descrição
id	UUID	PK	NÃO	Identificador único da leitura
device_id	UUID	FK	NÃO	Dispositivo que realizou a leitura
temperature_celsius	FLOAT	—	SIM	Temperatura do ambiente em graus Celsius
humidity_percent	FLOAT	—	SIM	Umidade relativa do ar em porcentagem (0-100)
soil_moisture_percent	FLOAT	—	SIM	Umidade do solo em porcentagem (0-100)
light_lux	FLOAT	—	SIM	Luminosidade ambiente em lux (se sensor disponível)
status	VARCHAR(20)	—	NÃO	Status calculado: "normal", "warning" ou "critical"
recorded_at	TIMESTAMP	—	NÃO	Timestamp exato da coleta pelo sensor

3.4 Tabela: irrigation_event

Registra cada acionamento da bomba de irrigação. O tipo de gatilho distingue ações automáticas (por lógica do sistema) de manuais (disparadas pelo usuário via app Angular). O campo trigger_reason detalha o motivo (ex: "Solo abaixo de 30%").

Campo	Tipo	Restrição	Null	Descrição
id	UUID	PK	NÃO	Identificador único do evento de irrigação
device_id	UUID	FK	NÃO	Dispositivo que executou a irrigação
trigger_type	VARCHAR(20)	—	NÃO	Origem do acionamento: "automatic" ou "manual"
duration_seconds	FLOAT	—	NÃO	Duração total da irrigação em segundos
water_volume_ml	FLOAT	—	SIM	Volume estimado de água utilizado (em ml)
trigger_reason	VARCHAR(300)	—	SIM	Descrição legível do motivo do acionamento
success	BOOLEAN	—	NÃO	Indica se a irrigação foi concluída com sucesso
started_at	TIMESTAMP	—	NÃO	Data e hora do início da irrigação
finished_at	TIMESTAMP	—	SIM	Data e hora do fim da irrigação

3.5 Tabela: plant_image

Armazena metadados e conteúdo das imagens capturadas pela câmera Canon integrada ao sistema. O campo `image_base64` guarda o conteúdo codificado, enquanto `storage_path` aponta para o arquivo salvo em disco ou armazenamento externo (recomendado para produção). Cada imagem é vinculada a uma análise de visão computacional.

Campo	Tipo	Restrição	Null	Descrição
id	UUID	PK	NÃO	Identificador único da imagem
device_id	UUID	FK	NÃO	Dispositivo/câmera que capturou a imagem
analysis_id	UUID	FK	SIM	Referência à análise de visão computacional associada
image_base64	TEXT	—	SIM	Conteúdo da imagem codificado em Base64
image_format	VARCHAR(10)	—	NÃO	Formato da imagem: "jpeg", "png" etc.
width_px	INTEGER	—	SIM	Largura da imagem em pixels
height_px	INTEGER	—	SIM	Altura da imagem em pixels
file_size_bytes	INTEGER	—	SIM	Tamanho do arquivo em bytes
storage_path	VARCHAR(500)	—	SIM	Caminho do arquivo em disco ou URL no armazenamento externo
captured_at	TIMESTAMP	—	NÃO	Data e hora da captura da imagem

3.6 Tabela: vision_analysis

Resultado do processamento de visão computacional (OpenCV + Python) sobre cada imagem capturada. O `health_score` é um índice de 0 a 100 que representa o estado geral da planta. O

campo `detected_issues` armazena um JSON com os problemas identificados (ex: manchas, amarelamento, pragas).

Campo	Tipo	Restrição	Null	Descrição
<code>id</code>	UUID	PK	NÃO	Identificador único da análise
<code>image_id</code>	UUID	FK	NÃO	Imagem que foi analisada
<code>model_version</code>	VARCHAR(30)	—	NÃO	Versão do modelo/algoritmo OpenCV utilizado
<code>health_score</code>	FLOAT	—	NÃO	Índice de saúde da planta de 0 (crítico) a 100 (excelente)
<code>health_status</code>	VARCHAR(20)	—	NÃO	Classificação: "excellent", "good", "warning", "critical"
<code>leaf_coverage_percent</code>	FLOAT	—	SIM	Percentual de cobertura foliar detectado na imagem (%)
<code>detected_issues</code>	TEXT	—	SIM	JSON com lista de problemas detectados e suas coordenadas
<code>recommendations</code>	TEXT	—	SIM	Sugestões geradas com base na análise da imagem
<code>confidence_score</code>	FLOAT	—	NÃO	Grau de confiança da análise (0.0 a 1.0)
<code>processing_time_ms</code>	INTEGER	—	SIM	Tempo de processamento da imagem em milissegundos
<code>analyzed_at</code>	TIMESTAMP	—	NÃO	Data e hora da conclusão da análise

3.7 Tabela: `system_event`

Log de auditoria de eventos do sistema: reconexões do ESP32, falhas de sensor, reinicializações, atualizações de firmware e outros eventos operacionais. Essencial para rastreabilidade e diagnóstico em apresentações e relatórios técnicos.

Campo	Tipo	Restrição	Null	Descrição
<code>id</code>	UUID	PK	NÃO	Identificador único do evento
<code>device_id</code>	UUID	FK	SIM	Dispositivo relacionado ao evento (nulo para eventos globais)
<code>event_type</code>	VARCHAR(50)	—	NÃO	Tipo do evento: "device_connected", "sensor_failure", "firmware_update", etc.
<code>severity</code>	VARCHAR(20)	—	NÃO	Gravidade: "info", "warning", "error", "critical"
<code>description</code>	VARCHAR(500)	—	NÃO	Descrição legível do evento ocorrido
<code>payload_json</code>	TEXT	—	SIM	Dados adicionais em formato JSON (parâmetros do evento)

Campo	Tipo	Restrição	Null	Descrição
acknowledged	BOOLEAN	—	NÃO	Indica se o evento foi reconhecido por um operador
occurred_at	TIMESTAMP	—	NÃO	Data e hora exata do ocorrido

3.8 Tabela: alert

Alertas gerados automaticamente quando uma leitura de sensor ultrapassa os thresholds definidos para a planta. Cada alerta registra o valor esperado e o valor real, além do timestamp de resolução quando normalizado.

Campo	Tipo	Restrição	Null	Descrição
id	UUID	PK	NÃO	Identificador único do alerta
device_id	UUID	FK	NÃO	Dispositivo que gerou o alerta
sensor_reading_id	UUID	FK	SIM	Leitura de sensor que disparou o alerta
alert_type	VARCHAR(30)	—	NÃO	Categoria: "temperature", "humidity", "soil_moisture", "connectivity"
metric_name	VARCHAR(50)	—	NÃO	Nome da métrica que disparou o alerta
threshold_value	FLOAT	—	NÃO	Valor limite configurado para a métrica
actual_value	FLOAT	—	NÃO	Valor real medido pelo sensor no momento do alerta
message	VARCHAR(300)	—	NÃO	Mensagem descritiva do alerta exibida no frontend
resolved	BOOLEAN	—	NÃO	Indica se o alerta foi resolvido
triggered_at	TIMESTAMP	—	NÃO	Data e hora em que o alerta foi disparado
resolved_at	TIMESTAMP	—	SIM	Data e hora em que o alerta foi resolvido (nulo se pendente)

4. Relacionamentos entre Entidades

A tabela a seguir descreve todos os relacionamentos do schema, com as cardinalidades e a lógica de negócio associada a cada vínculo:

Tabela origem	Tabela destino	Cardinalidade	Descrição
plant	device	1 : N	Uma planta pode ser monitorada por múltiplos dispositivos ESP32

Tabela origem	Tabela destino	Cardinalidade	Descrição
device	sensor_reading	1 : N	Um dispositivo gera múltiplas leituras de sensor ao longo do tempo
device	irrigation_event	1 : N	Um dispositivo executa múltiplos eventos de irrigação
device	plant_image	1 : N	Um dispositivo captura múltiplas imagens da planta
device	system_event	1 : N	Um dispositivo emite múltiplos eventos de sistema (log)
device	alert	1 : N	Um dispositivo gera múltiplos alertas ao longo do tempo
plant_image	vision_analysis	1 : 0..1	Cada imagem pode ter no máximo uma análise de visão associada
sensor_reading	alert	1 : N	Uma leitura de sensor pode disparar múltiplos alertas

5. Estratégia de Índices

Os índices a seguir foram definidos com base nos padrões de consulta esperados pelo backend Spring Boot e pelas queries analíticas do dashboard Angular. Todos os campos de chave estrangeira e as colunas de timestamp usadas em filtros temporais recebem índice:

Tabela	Coluna(s)	Justificativa
sensor_reading	device_id, recorded_at	Consultas de séries temporais por dispositivo
sensor_reading	recorded_at	Filtragem por intervalos de tempo
irrigation_event	device_id, started_at	Histórico de irrigação por dispositivo
plant_image	device_id, captured_at	Listagem de imagens cronológica por dispositivo
alert	device_id, resolved, triggered_at	Alertas ativos e histórico por dispositivo
system_event	device_id, occurred_at	Log de eventos por dispositivo
device	mac_address	Lookup rápido por identificador físico (único)
device	plant_id	Join frequente device → plant

6. Decisões de Design

6.1 UUID como chave primária

Todas as tabelas utilizam UUID (Universally Unique Identifier) como chave primária em vez de inteiros sequenciais. Isso garante que os IDs possam ser gerados pelo cliente (ESP32 ou aplicação) sem risco de colisão, facilita a distribuição horizontal futura e evita a exposição de sequências previsíveis nas APIs.

6.2 Imagens: Base64 vs armazenamento externo

O campo `image_base64` na tabela `plant_image` permite armazenar a imagem diretamente no banco para simplicidade de implementação e portabilidade em ambiente acadêmico. Para ambientes de produção, recomenda-se utilizar apenas o campo `storage_path`, apontando para um sistema de arquivos local (ex: `/var/plant-images/`) ou serviço de armazenamento em nuvem (AWS S3, Google Cloud Storage). Armazenar objetos binários grandes (BLOBs) diretamente em tabelas relacionais degrada o desempenho de consultas nas demais colunas.

6.3 Status calculado em `sensor_reading`

O campo `status` na tabela `sensor_reading` ("normal", "warning", "critical") é calculado no momento da inserção pelo backend, comparando os valores lidos com os parâmetros ideais da planta vinculada ao dispositivo. Essa abordagem desnormaliza levemente o banco em favor da performance: consultas de dashboard não precisam fazer joins com a tabela `plant` para filtrar por status.

6.4 Payload JSON em `system_event`

O campo `payload_json` na tabela `system_event` armazena metadados adicionais em formato JSON livre. Isso oferece flexibilidade para registrar parâmetros específicos de cada tipo de evento (ex: versão anterior e nova versão em um evento de `firmware_update`) sem necessidade de criar colunas para cada caso de uso.

6.5 Separação de alert e `system_event`

Alertas e eventos de sistema são armazenados em tabelas separadas por terem ciclos de vida distintos. Um alert tem estado (`resolved/triggered`) e deve ser exibido ativamente no dashboard até ser resolvido. Um `system_event` é um registro imutável de auditoria. Misturá-los na mesma tabela dificultaria consultas e a lógica de resolução de alertas.

7. Convenções de Nomenclatura

- Todos os nomes de tabelas e colunas utilizam `snake_case` em inglês
- Chaves primárias sempre nomeadas `id` (tipo UUID)
- Chaves estrangeiras seguem o padrão `{tabela_referenciada}_id`
- Colunas de timestamp utilizam sufixo `_at` (ex: `created_at`, `recorded_at`)

- Colunas booleanas utilizam prefixo `is_` ou são nomeadas com adjetivo (ex: `is_active`, `resolved`, `success`)
- Campos de porcentagem utilizam sufixo `_percent`; campos em Celsius, `_celsius`; em milissegundos, `_ms`

8. Scripts SQL — Criação das Tabelas

Script de criação do schema no PostgreSQL. Execute os comandos na ordem apresentada para respeitar as dependências entre chaves estrangeiras:

```
-- Habilitar extensão UUID
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

-- Tabela: plant
CREATE TABLE plant (
    id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    common_name       VARCHAR(100) NOT NULL,
    scientific_name    VARCHAR(150),
    description        TEXT,
    optimal_temp_min   FLOAT NOT NULL,
    optimal_temp_max   FLOAT NOT NULL,
    optimal_humidity_min   FLOAT NOT NULL,
    optimal_humidity_max   FLOAT NOT NULL,
    optimal_soil_moisture_min   FLOAT NOT NULL,
    optimal_soil_moisture_max   FLOAT NOT NULL,
    watering_interval_hours INTEGER NOT NULL,
    created_at         TIMESTAMP NOT NULL DEFAULT NOW()
);

-- Tabela: device
CREATE TABLE device (
    id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    plant_id          UUID NOT NULL REFERENCES plant(id),
    name              VARCHAR(80) NOT NULL,
    mac_address        VARCHAR(17) NOT NULL UNIQUE,
    firmware_version   VARCHAR(20),
    location_description VARCHAR(200),
    is_active          BOOLEAN NOT NULL DEFAULT TRUE,
    last_seen_at       TIMESTAMP,
    created_at         TIMESTAMP NOT NULL DEFAULT NOW()
);

-- Tabela: sensor_reading
CREATE TABLE sensor_reading (
    id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    device_id          UUID NOT NULL REFERENCES device(id),
    temperature_celsius   FLOAT,
    humidity_percent      FLOAT,
    soil_moisture_percent  FLOAT,
    light_lux            FLOAT,
    status              VARCHAR(20) NOT NULL DEFAULT 'normal',
```

```
    recorded_at          TIMESTAMP NOT NULL DEFAULT NOW()
);

-- Tabela: irrigation_event
CREATE TABLE irrigation_event (
    id                   UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    device_id            UUID NOT NULL REFERENCES device(id),
    trigger_type         VARCHAR(20) NOT NULL,
    duration_seconds     FLOAT NOT NULL,
    water_volume_ml      FLOAT,
    trigger_reason       VARCHAR(300),
    success              BOOLEAN NOT NULL DEFAULT TRUE,
    started_at           TIMESTAMP NOT NULL,
    finished_at          TIMESTAMP
);

-- Tabela: plant_image
CREATE TABLE plant_image (
    id                   UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    device_id            UUID NOT NULL REFERENCES device(id),
    analysis_id          UUID,
    image_base64         TEXT,
    image_format         VARCHAR(10) NOT NULL,
    width_px             INTEGER,
    height_px            INTEGER,
    file_size_bytes      INTEGER,
    storage_path         VARCHAR(500),
    captured_at          TIMESTAMP NOT NULL DEFAULT NOW()
);

-- Tabela: vision_analysis
CREATE TABLE vision_analysis (
    id                   UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    image_id             UUID NOT NULL REFERENCES plant_image(id),
    model_version        VARCHAR(30) NOT NULL,
    health_score         FLOAT NOT NULL,
    health_status        VARCHAR(20) NOT NULL,
    leaf_coverage_percent FLOAT,
    detected_issues      TEXT,
    recommendations      TEXT,
    confidence_score     FLOAT NOT NULL,
    processing_time_ms   INTEGER,
    analyzed_at          TIMESTAMP NOT NULL DEFAULT NOW()
);

-- Tabela: system_event
CREATE TABLE system_event (
    id                   UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    device_id            UUID REFERENCES device(id),
    event_type           VARCHAR(50) NOT NULL,
    severity             VARCHAR(20) NOT NULL,
    description          VARCHAR(500) NOT NULL,
    payload_json         TEXT,
    acknowledged         BOOLEAN NOT NULL DEFAULT FALSE,
```

```
    occurred_at    TIMESTAMP NOT NULL DEFAULT NOW()
);

-- Tabela: alert
CREATE TABLE alert (
    id              UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    device_id       UUID NOT NULL REFERENCES device(id),
    sensor_reading_id UUID REFERENCES sensor_reading(id),
    alert_type      VARCHAR(30) NOT NULL,
    metric_name     VARCHAR(50) NOT NULL,
    threshold_value  FLOAT NOT NULL,
    actual_value     FLOAT NOT NULL,
    message         VARCHAR(300) NOT NULL,
    resolved        BOOLEAN NOT NULL DEFAULT FALSE,
    triggered_at    TIMESTAMP NOT NULL DEFAULT NOW(),
    resolved_at     TIMESTAMP
);

-- Adicionar FK circular: plant_image -> vision_analysis
ALTER TABLE plant_image
    ADD CONSTRAINT fk_plant_image_analysis
    FOREIGN KEY (analysis_id) REFERENCES vision_analysis(id);
```

9. Scripts SQL — Criação dos Índices

```
-- Índices para sensor_reading (série temporal)
CREATE INDEX idx_sensor_device_time ON sensor_reading(device_id, recorded_at DESC);
CREATE INDEX idx_sensor_time ON sensor_reading(recorded_at DESC);

-- Índices para irrigation_event
CREATE INDEX idx_irrigation_device_time ON irrigation_event(device_id, started_at DESC);

-- Índices para plant_image
CREATE INDEX idx_image_device_time ON plant_image(device_id, captured_at DESC);

-- Índices para alert
CREATE INDEX idx_alert_device_status ON alert(device_id, resolved, triggered_at DESC);

-- Índices para system_event
CREATE INDEX idx_event_device_time ON system_event(device_id, occurred_at DESC);

-- Índice único no mac_address do device (já garantido pelo UNIQUE, explícito para clareza)
CREATE UNIQUE INDEX idx_device_mac ON device(mac_address);

-- Índice de join frequente device -> plant
CREATE INDEX idx_device_plant ON device(plant_id);
```

10. Considerações Finais

O schema apresentado foi projetado para atender tanto à fase acadêmica do projeto quanto a uma eventual evolução para produção. As principais escolhas arquiteturais — UUID como PK, separação de alertas e eventos, campo de status calculado, suporte a duas estratégias de armazenamento de imagens — foram tomadas com o objetivo de maximizar a rastreabilidade, a performance de consultas em série temporal e a clareza do código backend.

Para a apresentação acadêmica, recomenda-se demonstrar o fluxo completo: leitura de sensor → publicação MQTT → persistência no banco → exibição no dashboard Angular → geração de alerta automático quando o valor sai da faixa ideal configurada na tabela plant.